GieldaApi – etap 1: rozeznanie

Nov 6, 2026 · @Dawid

Zanim zaczniesz cokolwiek zmieniać, musisz wiedzieć, co się dzieje od momentu wysłania żądania w Postmanie do zapisu w bazie. Ten etap to 9 zadań, mniej więcej 4–5 wieczorów po 3 h. Kod zmieniasz dopiero w ostatnim.

Jak pracować

- Przy każdym zadaniu są pytania. Odpowiadaj na nie własnymi słowami, najlepiej w pliku `NOTATKI.md` w repo. Odpowiedzi do sprawdzenia są na końcu dokumentu.
- Najpierw szukaj w kodzie, potem pytaj mnie, a na końcu dopiero zaglądaj do odpowiedzi.
- Po każdym zadaniu zrób commit, np. `git commit -m "Zadanie 3: notatki o SQL"`. To nawyk z pracy, a do tego zawsze możesz wrócić do działającej wersji.
- Jeśli coś się wysypie, wklej mi cały błąd z konsoli. Czytanie błędów to też część nauki.

Zadanie 0 – uruchom projekt

Cel: API działa u Ciebie, a w Postmanie widzisz dane.

1. `dotnet --version` – wynik musi zaczynać się od `10.`. Jeśli nie: `brew install --cask dotnet-sdk`.
2. W głównym folderze repo: `dotnet tool restore`.
3. `dotnet ef migrations add InitialCreate --project src/GieldaApi --output-dir Data/Migrations`
4. `dotnet run --project src/GieldaApi` i poczekaj na `Now listening on: http://localhost:5080`.
5. Postman: zaimportuj `postman/GieldaApi.postman_collection.json` i wyślij „Lista spółek”.

Pytania:

- [ ] Ile spółek zwraca lista i ile wynosi `totalCount`?
- [ ] Jaki nowy plik pojawił się w `src/GieldaApi` po pierwszym uruchomieniu?
- [ ] Zatrzymaj API (Ctrl+C), usuń ten plik i uruchom ponownie. Co się stało z danymi?

Zadanie 1 – mapa projektu

Cel: wiesz, gdzie czego szukać. Otwórz każdy plik w `src/GieldaApi` i przeczytaj komentarze. Nie musisz rozumieć każdej linijki.

- [ ] Który plik uruchamia się jako pierwszy i co robi?
- [ ] Gdzie jest adres bazy danych (connection string)?
- [ ] Czym różni się `Domain/Company.cs` od `CompanyResponse` w `Contracts/` ? Po co dwie klasy o prawie tych samych polach?
- [ ] W którym pliku jest zapisane, że ticker ma maksymalnie 10 znaków i musi być unikalny?
- [ ] Narysuj na kartce strzałkami, kto kogo woła: kontroler, serwis, `AppDbContext`, baza.

Zadanie 2 – droga jednego żądania

Cel: śledzisz jedno żądanie od wejścia do odpowiedzi. Weź `GET /api/companies/PKO` i przejdź po kodzie krok po kroku.

- [ ] Która metoda w kontrolerze obsługuje to żądanie i skąd ASP.NET wie, że to ona?
- [ ] Która metoda serwisu jest wołana?
- [ ] Wyślij `GET /api/companies/pko` (małymi literami). Działa? Która linijka kodu za to odpowiada?
- [ ] Wyślij `GET /api/companies/XYZ`. Kto rzuca błąd, a kto zamienia go na odpowiedź 404?
- [ ] Dlaczego w kontrolerze nie ma żadnego `if`, który sprawdza, czy spółka istnieje?

Zadanie 3 – podejrzyj SQL

Cel: widzisz, na jaki SQL EF Core zamienia Twój LINQ. W trybie Development każde zapytanie do bazy wypisuje się w konsoli, w której działa `dotnet run`.

1. Wyczyść konsolę (Cmd+K w terminalu), wyślij `GET /api/companies?sector=Banki` i znajdź w logach linie z `Executed DbCommand`.
2. Zrób to samo dla `GET /api/companies?page=2&pageSize=3`.

- [ ] Ile zapytań SQL wykonuje jedno żądanie o listę? Dlaczego tyle?
- [ ] Który fragment SQL odpowiada `.Where(c => c.Sector == ...)`, a który `.Skip(...).Take(...)`?
- [ ] Czy zapytanie pobiera wszystkie kolumny tabeli, czy tylko część? Która linijka w `CompanyService` o tym decyduje?
- [ ] Ile zapytań wykonuje `GET /api/companies/PKO`? Znajdź je w kodzie.

Zadanie 4 – zajrzyj do bazy

Cel: widzisz tabele i dane bezpośrednio, bez API. Zainstaluj jedno z narzędzi: **DB Browser for SQLite** (`brew install --cask db-browser-for-sqlite`), rozszerzenie **SQLite Viewer** w VS Code albo okno Database w Riderze. Otwórz plik `src/GieldaApi/gielda.db`.

- [ ] Jakie tabele są w bazie? Której nie ma w kodzie jako encji?
- [ ] Co jest zapisane w tabeli `__EFMigrationsHistory`?
- [ ] Jakiego typu jest kolumna `Close` w tabeli `Quotes`? Czemu nie liczbowa?
- [ ] Jak w tabeli `Quotes` zapisane jest, do której spółki należy notowanie?
- [ ] Znajdź indeksy. Który pilnuje unikalnego tickera, a który jednego notowania dziennie?

Zadanie 5 – przeczytaj migrację

Cel: rozumiesz, czym jest migracja i co EF wygenerował za Ciebie. Otwórz folder `src/GieldaApi/Data/Migrations`.

- [ ] Ile plików jest w folderze i do czego służy każdy z nich?
- [ ] Co robi metoda `Up`, a co `Down`? Kiedy uruchomi się `Down`?
- [ ] Znajdź linijki, które tworzą unikalny indeks na tickerze. Porównaj z `CompanyConfiguration.cs`.
- [ ] Uruchom `dotnet ef migrations script --project src/GieldaApi`. Co dostajesz i do czego może się to przydać w pracy?
- [ ] Dlaczego migracji nie poprawia się ręcznie po tym, jak została nałożona na bazę?

Zadanie 6 – eksperymenty w Postmanie

Cel: wiesz, jak API reaguje na dobre i złe dane. Po każdym kroku sprawdź wynik także w podglądzie bazy z zadania 4.

1. Uruchom całą kolekcję: prawy klik na kolekcję → **Run collection**. Wszystko powinno być zielone.
2. Dodaj spółkę z tickerem pisanym małymi literami, np. `"ticker": "ten"`.
3. Dodaj notowanie z ujemnym wolumenem, a potem z datą w złym formacie, np. `"date": "03.10.2026"`.
4. Usuń spółkę, która ma notowania (np. `DELETE /api/companies/CDR`), a potem wyślij `GET /api/companies/CDR/quotes`.

- [ ] Jaki ticker zapisał się w bazie w kroku 2?
- [ ] Czym różnią się odpowiedzi 400 z kroku 3? Gdzie w kodzie powstaje każda z nich?

- [ ] Co się stało z notowaniami CDR w bazie po usunięciu spółki? Która linijka konfiguracji za to odpowiada?
- [ ] Odpal kolekcję jeszcze raz. Które testy się teraz wywalają i dlaczego? (Podpowiedź: usuń bazę jak w zadaniu 0, żeby wrócić do stanu początkowego.)

Zadanie 7 – debugger

Cel: zatrzymujesz program w trakcie działania i patrzysz, co jest w zmiennych. To najszybszy sposób na zrozumienie cudzego kodu, także w pracy.

1. Otwórz repo w Riderze albo VS Code z C# Dev Kit.
2. Postaw breakpoint (klik obok numeru linii) na pierwszej linijce `CompanyService.GetAllAsync`.
3. Uruchom w trybie debugowania (Rider: ikona robaka, VS Code: F5) i wyślij „Lista – filtr po sektorze".
4. Przechodź linijka po linijce (F10) i patrz na zmienne.

- [ ] Co jest w zmiennej `query` po zatrzymaniu?
- [ ] Jakiego typu jest zmienna `companies` przed `ToListAsync`? Co to oznacza? (Wróć do pytania bonusowego z LINQ seria 3.)
- [ ] W którym momencie w konsoli pojawia się SQL: przy `Where` czy przy `CountAsync`?

Zadanie 8 – pierwsza zmiana w kodzie

Cel: dodajesz własny endpoint, wzorując się na istniejącym kodzie. Bez migracji, bo nie zmieniasz bazy.

Nowy endpoint `GET /api/sectors` zwraca listę sektorów bez powtórzeń, alfabetycznie, np. `["Banki", "Gry", ...]`.

1. Dodaj metodę `GetSectorsAsync` do `ICompanyService` i zaimplementuj ją w `CompanyService`. LINQ znasz z serii 2.
2. Utwórz nowy kontroler `SectorsController` z jedną metodą `GET`. Wzoruj się na `CompaniesController`.
3. Dodaj żądanie do kolekcji w Postmanie, z testem na kod 200.
4. Sprawdź w konsoli, jaki SQL się wykonał.

- [ ] Czy sortowanie i usuwanie powtórzeń dzieje się w bazie, czy w pamięci aplikacji? Jak to sprawdzić?
- [ ] Czego zapomniałeś przy pierwszym podejściu? (Typowe: `CancellationToken`, `AsNoTracking`, rejestracja w DI – sprawdź, które z nich są tu w ogóle potrzebne.)

Po zadaniu 8 przyślij mi kod albo opisz, jak to zrobiłeś. Zrobię Ci code review jak w pracy i przejdziemy do etapu 2: zmiana bazy przez migrację i nowe pole w spółce.

Odpowiedzi – zaglądaj dopiero po swoich

Zadanie 0

- 8 spółek, `totalCount: 8`. Domyślny rozmiar strony to 20, więc wszystkie mieszczą się na pierwszej.
- Pojawia się `gielda.db` – cała baza SQLite to ten jeden plik.
- Po usunięciu i ponownym starcie baza powstaje od nowa: migracje tworzą tabele, a seeder wrzuca te same 8 spółek. Wszystko, co dodałeś sam, znika.

Zadanie 1

- `Program.cs`: rejestruje usługi w DI, buduje pipeline (obsługa błędów, kontrolery), a w trybie Development nakłada migracje.
- Connection string jest w `appsettings.json`: `"Default": "Data Source=gielda.db"`.
- `Company` to kształt tabeli w bazie (ma `Id` i listę notowań). `CompanyResponse` to kształt odpowiedzi API. Rozdzielenie sprawia, że zmiana w bazie nie psuje automatycznie API, a wewnętrzne pola, jak `Id`, nie wyciekają na zewnątrz.
- `Data/Configurations/CompanyConfiguration.cs`: `HasMaxLength(10)` i `HasIndex(...).IsUnique()`.
- Strzałki: Kontroler → Serwis → `AppDbContext` → SQLite. Kontroler nigdy nie rozmawia z bazą bezpośrednio.

Zadanie 2

- `CompaniesController.GetByTicker`. Ścieżka składa się z `[Route("api/companies")]` na klasie i `[HttpGet("{ticker}")]` na metodzie.
- `CompanyService.GetByTickerAsync`.
- Działa, bo metoda `Normalize` zamienia ticker na wielkie litery (`ToUpperInvariant`).
- Błąd rzuca serwis (`NotFoundException`). Na 404 zamienia go `GlobalExceptionHandler`, włączony w `Program.cs` przez `AddExceptionHandler` i `UseExceptionHandler`.
- Bo błędy obsługiwane są w jednym miejscu. Kontroler zajmuje się tylko ścieżką, w której wszystko się udaje.

Zadanie 3

- Dwa: `COUNT` do `totalCount` i `SELECT` z wynikami strony.
- `Where` to `WHERE "c"."Sector" = @...`, a `Skip` / `Take` to `LIMIT ... OFFSET ...` na końcu zapytania.
- Tylko 4 kolumny (`Ticker`, `Name`, `Sector`, `CreatedAtUtc`). Decyduje o tym `.Select(c => new CompanyResponse(...))`, czyli projekcja do DTO.
- Dwa: spółka razem z liczbą notowań oraz osobno ostatnie notowanie. Oba są w `GetByTickerAsync`.

Zadanie 4

- `Companies`, `Quotes`, `__EFMigrationsHistory` i systemowa `sqlite_sequence` (licznik autonumeracji `Id`). Tabeli historii migracji nie ma w kodzie – tworzy ją EF.
- Lista nałożonych migracji (`MigrationId`) i wersja EF, która je nałożyła. Dzięki temu EF wie, które migracje już były.
- `TEXT`. SQLite nie ma typu dziesiętnego, więc EF zapisuje `decimal` jako tekst, żeby nie stracić dokładności. Dlatego w SQLite nie posortujesz po cenie po stronie bazy (patrz README).
- Kolumna `CompanyId` to klucz obcy wskazujący `Id` w tabeli `Companies`.
- `IX_Companies_Ticker` (unikalny ticker) i `IX_Quotes_CompanyId_Date` (jedno notowanie dziennie).

Zadanie 5

- Trzy pliki: `..._InitialCreate.cs` (sama migracja), `..._InitialCreate.Designer.cs` (stan modelu w chwili tej migracji) i `AppDbContextModelSnapshot.cs` (aktualny stan modelu). Snapshot służy do wyliczenia różnicy przy następnej migracji.
- `Up` nakłada zmiany (tworzy tabele), `Down` je cofa (usuwa tabele). `Down` uruchomi się np. przy `dotnet ef database update 0` albo przy cofaniu do wcześniejszej migracji.
- `migrationBuilder.CreateIndex(name: "IX_Companies_Ticker", ..., unique: true)` – to tłumaczenie `HasIndex(c => c.Ticker).IsUnique()`.
- Dostajesz czysty skrypt SQL. W wielu firmach na produkcję trafia właśnie taki skrypt: przegląda go zespół albo puszcza pipeline.
- Bo bazy, które już ją mają (Twoja, kolegów, produkcja), nie wykonają jej drugi raz i zostaną w starej wersji. Zmianę robi się nową migracją.

Zadanie 6

- `TEN` – serwis zamienia ticker na wielkie litery przed zapisem.
- Ujemny wolumen: błąd walidacji z polem `Volume` w `errors`, z atrybutu `[Range]` w `QuoteContracts.cs`. Zła data: błąd odczytu JSON (pole w stylu `$.date`), bo tekst nie da się zamienić na `DateOnly` jeszcze przed walidacją. Oba 400 zwraca automatycznie `[ApiController]`.
- Notowania CDR zniknęły razem ze spółką. Odpowiada za to `.OnDelete(DeleteBehavior.Cascade)` w `CompanyConfiguration.cs`, a w migracji widzisz to jako `onDelete: ReferentialAction.Cascade`. `GET` notowań zwraca 404.
- Pada „Dodaj notowanie” (409), bo notowanie PKO z 2026-10-06 już istnieje po pierwszym przebiegu. Takie testy zależą od stanu bazy – w etapie 2 zobaczysz, jak to rozwiązuje się w testach integracyjnych.

Zadanie 7

- `CompanyQuery { Sector = "Banki", Page = 1, PageSize = 20 }` – ASP.NET złożył to z query stringa.
- `IQueryable<Company>` (debugger może pokazać `EntityQueryable` albo `InternalDbSet`). To przepis na zapytanie, który jeszcze nie trafił do bazy.
- Przy `CountAsync`. `Where` tylko dopisuje warunek do przepisu – dokładnie jak w pytaniu bonusowym z serii 3.

Zadanie 8

Przykładowe rozwiązanie:

```
// ICompanyService
Task<IReadOnlyList<string>> GetSectorsAsync(CancellationToken ct);

// CompanyService
public async Task<IReadOnlyList<string>> GetSectorsAsync(CancellationToken
ct)
    => await db.Companies
        .Select(c => c.Sector)
        .Distinct()
        .OrderBy(s => s)
        .ToListAsync(ct);

// Controllers/SectorsController.cs
[ApiController]
[Route("api/sectors")]
public sealed class SectorsController(ICompanyService companies) :
```

```
ControllerBase
{
    [HttpGet]
    public async Task<ActionResult<IReadOnlyList<string>>>
GetAll(CancellationToken ct)
        => Ok(await companies.GetSectorsAsync(ct));
}
```

- W bazie: w logu SQL widzisz `DISTINCT` i `ORDER BY`. Gdybyś dał `ToListAsync()` przed `Distinct()`, baza zwróciłaby wszystkie wiersze, a reszta działałaby w pamięci.
- `CancellationToken` – potrzebny, przekaż go do `ToListAsync`. `AsNoTracking` – niepotrzebne, bo pobierasz same napisy, a nie encje, więc EF niczego nie śledzi. Rejestracja w DI – niepotrzebna: kontrolery ASP.NET znajduje sam, a serwis jest już zarejestrowany.
- Drobiazg na później: SQLite domyślnie sortuje po bajtach, więc polskie litery, np. „Ś”, mogłyby wylądować za „Z”. W naszych danych to nie występuje.